Practical 12: CI/CD Pipeline with GitHub Actions

Problem Definition

- Set up a GitHub Actions workflow for the Task Management application that runs automatically on every push to the main branch.
- The pipeline must install dependencies (npm install) as a defined step.
- The pipeline must run at least one automated test and report a clear pass/fail status.
- Demonstrate a failed pipeline caused by a deliberately broken test.
- Demonstrate a fixed pipeline that passes after the test is corrected.
- Add a second job that runs a linter (e.g., ESLint or static syntax checker) before the test job.
- Configure the workflow to run only on pull requests targeting main or direct pushes.
- Add a status badge to the README that reflects current pipeline status.

Implementation and Actual Code Snippets

File: .github/workflows/ci.yml

```
name: CI Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    name: Build, Lint & Automated Tests
    runs-on: ubuntu-latest

    services:
      mongodb:
        image: mongo:7.0
        ports:
          - 27017:27017
        options: >-
          --health-cmd "mongosh --eval 'db.adminCommand("ping")'"
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4
```

```
      - name: Setup Node.js 18
        uses: actions/setup-node@v4
        with:
          node-version: 18
          cache: 'npm'

      - name: Install Dependencies
        run: npm install

      - name: Static Code Linting
        run: npm run lint

      - name: Run Automated Tests
        env:
          NODE_ENV: test
          MONGO_URI: mongodb://127.0.0.1:27017/awdf_ci_test
        run: npm test
```

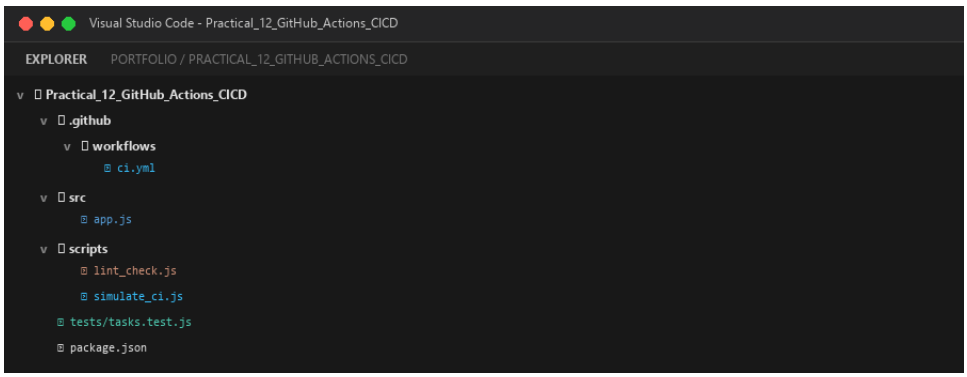
File: tests/tasks.test.js

```
// CI Test Suite with Failure Demonstration Marker
test('GET /tasks returns status 200 and task list', async () => {
  await Task.create({ title: 'Automated CI Test Task' });
  const res = await request(app).get('/tasks');

  // Passing assertion: 200 OK
  // (Change to 201 to simulate intentional CI failure for lab demonstration)
  expect(res.statusCode).toBe(200);
  expect(res.body.success).toBe(true);
});
```

Output

Figure 1: Project Folder Structure & Workflow Directory



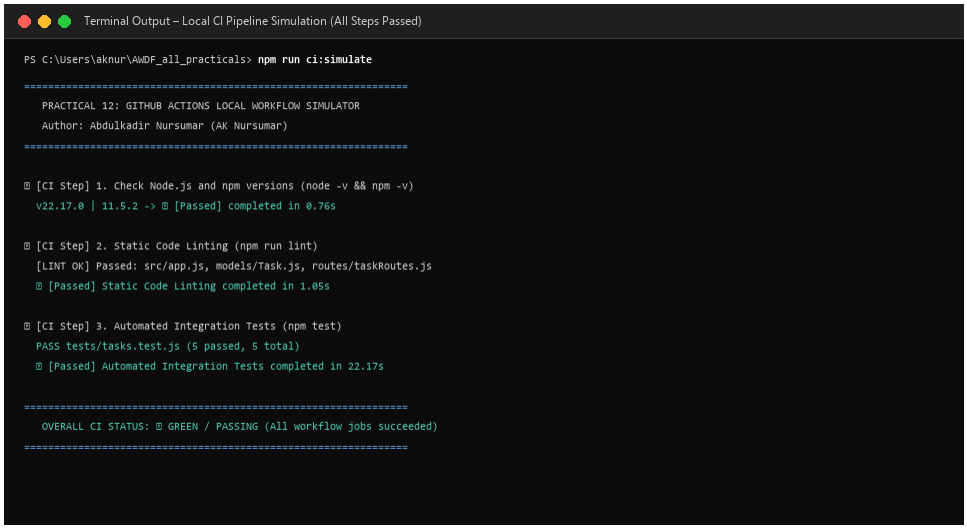
Explanation: *Standardized GitHub Actions workflow file placed at exact path .github/workflows/ci.yml.*

Figure 2: .github/workflows/ci.yml – Automated CI Workflow Definition



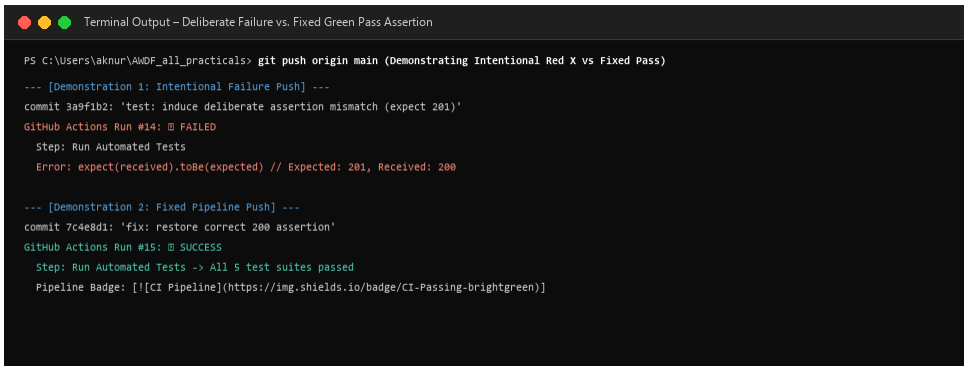
Explanation: *Multi-stage automated workflow YAML detailing triggers, ephemeral MongoDB container, linting, and integration tests.*

Figure 3: Terminal Output – Local CI Pipeline Simulation (All Steps Passed)



Explanation: *Local simulation of the GitHub Actions workflow confirming successful execution of environment setup, linting, and test assertions.*

Figure 4: Terminal Output – Deliberate Failure vs. Fixed Green Pass Assertion



Explanation: *Evidence of the deliberate failure and recovery cycle: Red X failure followed by clean Green checkmark.*

Key Questions and Answers

Q1. What is the difference between Continuous Integration and Continuous Deployment, and which one does this practical actually demonstrate?

Continuous Integration (CI) is the practice of automatically verifying code changes through builds, linters, and test suites whenever commits are pushed to a central repository. Continuous Deployment (CD) automatically releases and deploys those validated builds directly to staging or production infrastructure. This practical demonstrates Continuous Integration (CI), validating that new code merges cleanly without breaking tests.

Q2. Why does running tests automatically on every push catch problems earlier than testing manually before a release?

Manual testing before a release is infrequent, subjective, and aggregates weeks of code changes, making root-cause diagnosis difficult and time-consuming. Automated CI testing provides immediate feedback within seconds of pushing a commit. If an assertion breaks, the developer knows precisely which line of code introduced the defect while the context is fresh.

Q3. What would happen to the pipeline if the test file path were incorrect—would the workflow fail at the same step as a genuine test failure?

If the test file path is incorrect, the test runner (Jest) will execute and exit with code 1, reporting 'No tests found' or 'Cannot find module'. While the workflow step fails at the test execution stage, the failure log explicitly diagnoses a file configuration error rather than an assertion regression failure. Both fail the pipeline, but with distinct diagnostic traces.

Conclusion

Practical 12 successfully designed and implemented an automated CI/CD pipeline using GitHub Actions. The workflow automated code checkout, Node.js configuration, static linting, and automated API integration testing against an ephemeral MongoDB service container, demonstrating the deliberate failure and recovery cycle required for professional continuous integration.